System architecture

Proposed service boundaries and the path from scientific sources to the browser.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided



Browser → imagery / tile CDN • Batch sources → processing jobs → storage + database

01

Browser application

React + TypeScript + CesiumJS proposed. Global discovery leads into a detailed Jezero Marswalk. Local layers, science stops, round-trip planning and a field view are central. Scientific goal remains null until selected.

02

Static hosting / CDN

Serve compiled frontend assets through HTTPS. Browser requests imagery and 3D tiles directly from approved providers or object storage. Keep bulk tile traffic away from the API process.

03

Application API

Python + FastAPI proposed. Validate requests, search the catalog, return evidence, apply mission constraints and calculate lightweight rankings. Publish a versioned OpenAPI contract.

04

Spatial database

PostgreSQL + PostGIS proposed. Store crater records, footprints, scientific observations, dataset versions and analysis results. Use spatial indexes and explicit Mars coordinate reference systems.

05

Processing jobs

Python + GDAL proposed. Import source data and precompute regional terrain features. Use a separate batch process for heavy raster work; persistent queued jobs can be added when interactive routing needs them.

06

Scientific assets

Versioned object storage for raw snapshots, derived rasters and display tiles. NASA / USGS / PDS are source systems, not databases we assume are always reachable. Cesium ion is an optional visual terrain provider.

Read technical-brief.md for linked sources, assumptions and competition timing.

Frontend & API contracts

Responsive globe navigation with explicit request boundaries and empty states.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Frontend modules

GlobeView: global discovery and local Jezero view. MissionPanel: base, crew profile, limits and goal. LayersPanel: imagery, terrain and geology. StopPanel: tasks and evidence. FieldView: numbered stops, route and checklist.

02

State and accessibility

Walking astronauts are the primary workflow; crew vehicles and robotic support are optional profiles. Goal starts null. Keep map, mission and cached-data state separate. Support keyboard navigation, stale-request cancellation and explicit loading/error states.

03

Catalog endpoints — proposed

GET /v1/craters for bounded global search; GET /v1/craters/{id} for context; GET /v1/regions/jezero/stops for local science stops; GET /v1/layers for coverage and provenance. Support bbox filters and cursor pagination.

04

Analysis endpoints — proposed

POST /v1/rankings compares local stops using base, goal_id, crew/support profile and limits. Return component scores, planned science tasks, explanations and versions. Missing goal blocks ranking but not browsing. No global traversability claim.

05

Route jobs — later extension

POST /v1/routes creates a bounded analysis job and returns 202 + job_id. GET /v1/jobs/{id} reports progress; GET /v1/routes/{id} returns completed results. Define timeouts, cancellation, expiry and retry-safe submission IDs.

06

Browser performance

Cluster or tile markers at low zoom and fetch details on demand. Debounce searches, cache immutable assets and preserve attribution. Provide a list-based fallback when WebGL is unavailable. Do not download the full catalog into every browser.

Read technical-brief.md for linked sources, assumptions and competition timing.

Scientific data pipeline

Discover → snapshot → validate → normalize → derive → review → publish.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Source registry and ingestion

Select specific products from multiple NASA missions for Jezero: candidate sources include MRO imagery, MGS topography and Perseverance observations. Verify coverage and alignment. Record product IDs, citations, timestamps, licenses and checksums.

02

Planetary coordinate handling

Record body, CRS, latitude type, longitude direction/range, height datum and units for each product. Normalize explicitly; handle the longitude seam and poles. Do not label Mars positions EPSG:4326 or use Earth distances by default.

03

Raster processing

Retain original DEM and imagery. Produce validated analysis rasters and separate display tiles. GDAL COG is a proposed storage format; COG is not a ready-to-use Cesium terrain tileset. Derive slope only with verified horizontal and vertical units.

04

Core data entities

craters/regions: discovery context. stops: verified coordinates, science tasks, tools and work time. datasets/observations: provenance, resolution and values. excursions: base, ordered stops and return. runs: input, model and dataset versions.

05

Coverage and quality gates

Check duplicates, units, coordinate transforms, nodata, raster alignment and valid geometry. Treat footprints and pixel resolution as eligibility inputs. Sample-level coordinates remain pending until verified; a rover location is not automatically a drill location.

06

Serving and refresh

Publish reviewed dataset versions atomically. Store large rasters outside database rows; index footprints and metadata in PostGIS. Serve prebuilt display tiles via storage/CDN. Refresh deliberately, retaining old versions for reproducible rankings and rollback.

Read technical-brief.md for linked sources, assumptions and competition timing.

Ranking & route analysis

An explainable decision-support model; scientific goal and weights remain open.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Required mission inputs

Jezero base, selected goal, walking crew profile, optional vehicle/robot support, time/range budget and return reserve. Include carried equipment, science work time and crew configuration. All capabilities are versioned assumptions awaiting agreement.

02

Candidate filtering

Rank reachable science stops within verified Jezero coverage, not distant crater centers. Stops may be outcrops, deposits or viewpoints. Verify coordinates before placement. Exclude stops outside round-trip limits or without required approach data.

03

Separate score components

Science value: evidence relevant to the chosen goal. Travel cost: route length, time and elevation change. Terrain difficulty: mapped slopes and obstacles. Data confidence: coverage and resolution; show it separately, not as a probability of safety.

04

Priority model — not yet defined

Normalize comparable features with fixed documented bounds. Version weights and feature definitions. Show each contribution and exclusion. Do not choose weights, thresholds or goals silently. Missing data stays unknown and must not improve a score.

05

Route engine — staged scope

Marswalk planning includes base-to-stops-to-base travel, stop work time, terrain concerns and assumptions. Use clearly labeled distance estimates first; bounded grid routing comes later. Field view shows numbered stops, science tasks and reference imagery.

06

Scientific validation

Check known examples, projection math, nodata behavior and budget exclusions. Test whether rankings change sensibly with goals and profiles. Evaluate sensitivity to weights and data resolution. No hazard probability, validated human route or certified safety rating is claimed.

Read technical-brief.md for linked sources, assumptions and competition timing.

Hosting & server management

A small managed deployment first, with explicit responsibilities and cost controls.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Proposed deployment topology

Static frontend on a CDN; API in a managed Linux container; managed PostgreSQL/PostGIS; object storage for assets. Run imports as separate batch jobs. Add a dedicated worker and durable queue only when long-running route workloads justify them.

02

Runtime management

Pin dependencies and build immutable images. Configure process startup, health checks, restart policy and graceful shutdown. Size API concurrency around memory and database connections. Offload CPU-heavy terrain work so it cannot block catalog requests.

03

Environment and networking

Separate local development, staging and production configuration. Keep database private with least-privilege credentials. Use managed TLS termination and explicit allowed origins. Record provider, region, domain, monthly budget and ownership as team decisions.

04

Monitoring and incident handling

Capture structured logs with request/job IDs and dataset/model versions. Track API errors, latency, job failures, queue depth, storage and database health. Alert on persistent failures. Keep a short runbook: triage, pause jobs, rollback and restore.

05

Backups and recovery

Schedule database backups and define retention plus recovery targets with the team. Test restoring into an isolated environment. Version scientific assets; keep rebuild scripts and source checksums. Persist jobs and state outside ephemeral container filesystems.

06

Capacity and cost management

Cap query area, candidate count, raster resolution, job runtime and concurrent jobs. Use cache keys containing mission and dataset versions. Watch database, storage, bandwidth and imagery-provider quotas. Keep a small precomputed demo dataset for upstream outages.

Read technical-brief.md for linked sources, assumptions and competition timing.

Security, delivery & open decisions

Development ownership, verification, release flow and handoff.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

01

Security boundaries

Keep source-access credentials and database passwords on the server. Any browser imagery token must be deliberately public, scoped and origin-restricted where supported. Use parameterized SQL, request size limits and an allowlist for remotely fetched source URLs.

02

Identity and saved work

Default proposal: anonymous browsing with browser-local mission drafts and JSON export. Shared plans/accounts are optional, not assumed. If added, enforce server-side authorization on every saved object. Authenticate ingestion and administrative operations.

03

Repository and local setup

Proposed layout: frontend/, backend/, pipelines/, data/manifests/, infra/, tests/ and docs/. Commit schema migrations, example environment variables and lockfiles. Keep raw scientific data, credentials and generated bulk assets out of Git.

04

Checks and release pipeline

Pull request → build frontend → backend tests → scientific-data checks → staging → smoke test → release. Test mission validation, coordinate transforms, score invariants, source provenance and job failures. Store immutable artifacts and support code plus schema rollback plans.

05

Phased delivery

1: verify Jezero sources and stops; agree the scientific goal. 2: build local layered map and evidence panels. 3: compare stops and review a round-trip excursion. 4: add field view and global discovery. No application or infrastructure is deployed.

06

Decisions awaiting the team

Accepted: Jezero Marswalk centerpiece; global crater discovery supports it. Still open: scientific goal/weights, exact base and stops, crew/support profiles, stack acceptance, hosting budget and accounts. Ask before implementation that depends on these choices.

Read technical-brief.md for linked sources, assumptions and competition timing.